MiniDB: файловая мини-СУБД

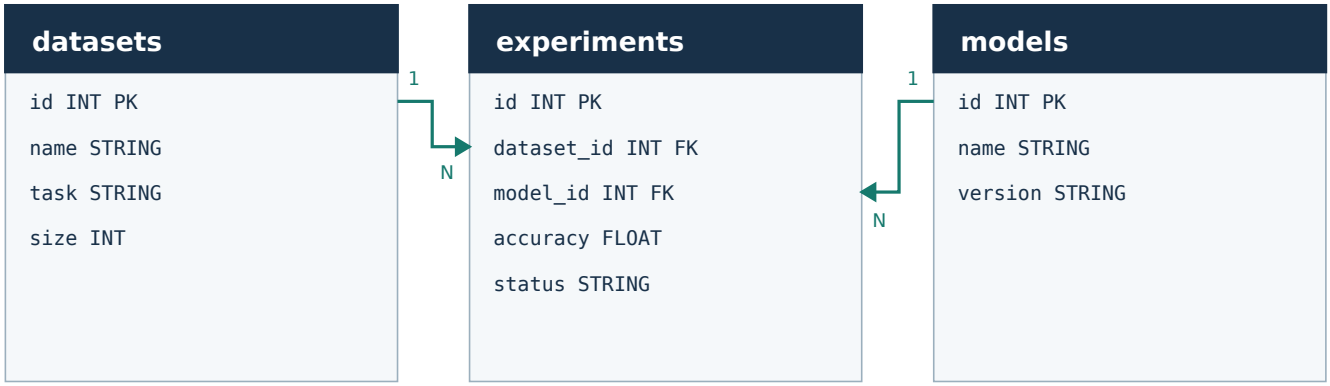
Лабораторная работа № 1 • Отчёт о текущей версии • 24.10.2026

1. Назначение и реализация

Разработана простая CLI-программа на Python 3.8+ для хранения датасетов, моделей и экспериментов. Приложение использует только стандартную библиотеку Python; готовые СУБД, ORM и JSON-хранилище не применяются. Реализованы CREATE, INSERT, массовая вставка, SELECT с одним условием, UPDATE и DELETE.

Архитектура: main.py отвечает за команды и вывод; database.py - за CRUD, типы, PK/FK и файлы; seed.py создаёт воспроизводимые данные; demo.py запускает сценарий защиты. Исходные тестовые данные: 5 datasets, 5 models и 20 experiments.

2. Схема данных



Один датасет и одна модель могут участвовать во многих экспериментах. Диаграмма построена локально по реализованной схеме; исходник для импорта в dbdiagram: docs/schema.dbml.

3. Формат хранения и запись

База - каталог с парами файлов «таблица.schema» и «таблица.csv». Схема: CSV-строки с именем поля, типом и отметкой PK. Данные: CSV без заголовка, UTF-8. Запятые, кавычки и переносы строк обрабатывает модуль csv.

```
models.schema: id,INT,PK / name,STRING, / version,STRING,
models.csv: 1,Linear Model,1.0
```

Чтение выполняется последовательным проходом. При записи итоговая таблица сначала проверяется в памяти, сохраняется во временный файл в каталоге базы и заменяет исходный файл. Ошибочная операция не записывает частичный результат. Данные и схема доступны после выхода и нового запуска.

Для id:INT:PK при пропуске ID назначается максимум текущих и ручных ID пакета + 1, начиная с 1. После удаления максимального ID возможен повтор номера. В массовой CLI-вставке используется флаг --auto-id.

Проверка и демонстрация

MiniDB • Подготовка к защите

4. Проверки целостности

Типы INT, STRING и FLOAT проверяются до записи. PK должен быть уникальным. Вставка и изменение experiments проверяют существование dataset_id и model_id. Нельзя удалить родителя или изменить его ID, если на него ссылается эксперимент. Каскадных операций нет: сначала удаляют или переназначают зависимые эксперименты.

5. Примеры операций

```
insert models "Новая модель" v1
select experiments where accuracy >= 0.9
update experiments set status completed where id = 21
delete experiments where id = 20
```

Вокруг оператора условия нужны пробелы. Строки с пробелами заключаются в кавычки. Без where UPDATE и DELETE обрабатывают все записи таблицы.

6. Результаты проверки

Проверка	Результат
python3 -m unittest discover -s tests -v	41 тест, все пройдены
CRUD, типы, 6 операторов SELECT, авто-ID	Проверены обычные и ошибочные операции
PK/FK и массовые изменения	Ошибочные операции отменяются целиком
Сбой замены файла; выход и EOF	Исходные данные сохранены; повторное открытие работает
python3 demo.py: два процесса CLI	7 ожидаемых ошибок PK/FK; конечное состояние проверено

Сценарий создаёт базу 5/5/20, таблицу notes, датасет 6, модель 6, две модели пакетом и эксперимент 21. Затем изменяет статус и имя датасета, удаляет эксперимент 20 и показывает ошибки PK/FK. После exit новый процесс читает experiment 21 со статусом completed; experiment 20 отсутствует. Итог: 6 datasets, 8 models, 20 experiments и 1 notes.

Команды: docs/demo_session1.txt и docs/demo_session2.txt. Инструкция: docs/DEMO.md. Протокол контрольного запуска: docs/demo-output.txt. Автоматический показ использует временную базу и не изменяет данные из поставки.

7. Ограничения и benchmark индекса

Индекс id → offset в текущей версии не реализован. Поиск выполняется полным проходом, поэтому benchmark «до/после индекса» не проводился и ускорение не заявляется. Этот пункт относится к следующему расширению проекта.

Также отсутствуют выбор отдельных столбцов, AND/OR, JOIN, backup/restore и SQL-парсер. Версия рассчитана на небольшой объём данных и один процесс записи. Замена файла не заменяет журнал восстановления и не гарантирует сохранность при отключении питания.